Kan we strike Oil? (Yes)

Muhammad Abdul Rehman Baig [*1] **Zindan Kurt** [*2]

Abstract

We aim to apply modern deep learning techniques to stock trading, and compare the performance against a novel and, as of yet unproven, new architecture called Komolgorov Arnold Networks. These new networks have a lot of theoretical power, but remain largely unexplored in practical applications. This project aims to compare their performance in a transformer model against standard MLP-based transformers. We have constructed a transformer architecture that is fine tuned for time series analysis and bench marked it on raw market data from Borsa Istanbul that we cleaned and engineered ourselves. We use a time2vec encoding in our model to aid it in recognizing periodic and non periodic trends in the data.

1. Introduction

Komolgorov Arnold Networks (KANs, as they will be referred to henceforth), are a relatively new introduction in the literature. These alternatives to MLPs, use families of learnable activation functions instead of weights and are theoretically much more powerful at function representation, as opposed to MLPs which only serve as function approximators. This can be best seen when comparing their respective mathematical foundations. First, consider the statement of the universal approximation for MLPs, for any continous discriminatory function σ, finite sums of the form

$$G(x) = \sum_{j=1}^{N} \alpha_j \sigma(\mathbf{w}_j^T \mathbf{x} + b_j), \text{ where } \mathbf{w}_j \in \mathbb{R}^n, \alpha_j, b_j \in \mathbb{R}$$

are dense in $C(I_n)$.

In other words, given any $\varepsilon > 0$ and $f \in C(I_n)$, there is a sum $G(x)$ of the above form such that

$$|G(x) - f(x)| < \varepsilon, \quad \forall x \in I_n.$$

Compare this against the Komolgorov representation theorem which states that for any function $f : [0, 1]^n \to \mathbb{R}$ there exist functions $\phi_{p,q} : [0, 1] \to \mathbb{R}$ and $\Phi_q : \mathbb{R} \to \mathbb{R}$ such that

$$f(\mathbf{x}) = f(x_1, \ldots, x_n) = \sum_{q=0}^{2n} \Phi_q \left(\sum_{p=1}^{n} \phi_{q,p}(x_p) \right)$$

While the mathematical details of these theorems deserve a discussion of their own, observe that the Universal Approximation Theorem displays a limiting behaviour, while the Komolgorov Representation Thoeorem gaurantees us an exact equality given a certain number of 'neurons'. It is this important difference between the two theorems that gives the KANs their higher representation power and makes them an intriguing area of research. In practice KANs, employ a family of parameterized and differentiable functions with several layers (Liu et al., 2024) and can be thought of as a generalization of MLP networks.

Our aim was to incorporate these networks in a transformer architecture and report on their performance differences on time series stock data analysis, when stacked against a vanilla transformer. We created these models from scratch, and chose to integrate KANs into transformers by swapping out the projection layer in a multi-head-self-attention module. We chose to engineer the stock data ourselves for greater control, instead of relying on pre-engineered datasets like Yahoo Finance. Feature engineering was a huge part of our workload, and it helped enhance the results we obtained by a considerable margin. These features, engineered from the dataset itself, can be thought of as kernel transformations. Intuitively, they lift the input space into a higher dimension, by combining the inputs in a non-linear way, which gives the model more space to work with.

We postulated that the stronger representation power of KANs would enable them to learn patterns and features in volatile stock data that traditional MLPs would struggle to approximate, or at least take a lengthy time to learn.

2. Related Work

There has been some academic attention given to KANs despite how recent they are, although the original paper (Liu et al., 2024) provides a superb implementation, by their own admittance, they did not attempt at making the models fast. Addressing these concerns of speed, we have (Yang & Wang, 2024) who attempted to leverage the better

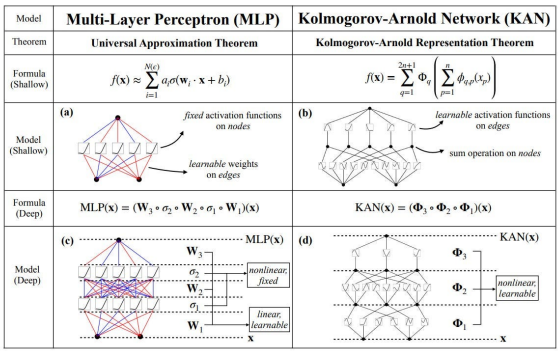

Figure 1. KANs vs MLPs

compatibility of rational functions in cuda to make KANs much faster. Their implementation was originally meant to be used in Image Transformers, however it works just as well in vanilla transformers with no changes required. On the financial side of the literature, we were inspired by (Muhammad et al., 2023) who suggested using a time2vec embedding in our transformer to make its predictions more robust. We also referred to (Chen et al., 2017) for their simple baseline LSTM model.

3.1. Dataset

We utilized a high-frequency market data processing framework specifically designed to handle data from the Borsa Istanbul (BIST) exchange, though it remains compatible with other markets. The dataset comprises of raw network capture (PCAP) files and exchange-specific virtual raw data (VRD) formats, which are transformed into standardized ITCH protocol messages for consistency. This allows us to conduct advanced market microstructure analysis.

The data acquisition and conversion layer processes these raw inputs (from January 2023 to March 2024), implementing the MoldUDP64 protocol for market data transmission and handling data compression for efficient storage. We used various validation mechanisms to maintain consistency. The dataset spans multiple financial instruments, including equities, futures, options, and warrants, each with specific data handling requirements.

Order book reconstruction is performed through a specialized engine that updates market states in real-time with nanosecond precision. This supports tracking top-of-book changes and handling multiple security types with distinct price quotation mechanisms.

Additionally, the dataset supports configurable time series generation at frequencies ranging from microseconds to minutes. Event-driven analysis tracks market state changes, and performance metrics such as execution time and resource usage are monitored throughout. Outputs include structured ITCH messages, top-of-book changes, and time series data, all organized hierarchically by data type, security class, and time period.

3.2. Feature Engineering for High-Frequency Market Data

We utilize a comprehensive feature engineering framework to process and analyze high-frequency market data, specifically designed for the Borsa Istanbul (BIST) equity market. The framework focuses on Turkish Airlines stock (THYAO), capturing second-level "Best Bid and Offer" (BBO) data from January 2023 to March 2024. This dataset forms the foundation for training our models.

3.2.1. Dataset Overview

The dataset comprises the following columns:

- **time**: Timestamp in HH:MM:SS format.
- **symbol**: Stock symbol, e.g., THYAO.
- **bid_no_of_order**: Number of orders at the best bid price.
- **bid_quantity**: Total volume available at the best bid price.
- **bid_price**: Highest price buyers are willing to pay.
- **ask_price**: Lowest price sellers are willing to accept.
- **ask_quantity**: Total volume available at the best ask price.
- **ask_no_of_order**: Number of orders at the best ask price.

Each record provides a snapshot of the order book state, capturing trading activity, liquidity, and price movements. The spread, calculated as Spread $=$ ask_price $-$ bid_price, is a critical measure for assessing trading costs and liquidity. For example, a consistent spread of 0.1 with stable order book depth suggests a liquid market for THYAO during this period.

3.2.2. Price Features

Key price features derived from the data include:

- **Open Price** ($P_{\textbf{open}}$): Represents the first price recorded in the time period. It serves as an indicator of market sentiment at the beginning of the trading session and is used to identify gaps between periods, which are crucial for day traders.

- **High Price (P_{high})**: The maximum price within the time period, reflecting maximum buying pressure. It is important for identifying resistance levels and understanding price momentum.
- **Low Price (P_{low})**: The minimum price recorded in the time period, representing maximum selling pressure. This feature is critical for risk management and volatility calculations.
- **Close Price (P_{close})**: The last price in the time period, widely used for technical analysis and representing the final market consensus for the period.
- **Mid Price (P_{mid})**: Calculated as:

$$P_{\text{mid}} = \frac{\text{bid\_price} + \text{ask\_price}}{2}.$$

  It provides a noise-free estimate of the market value, reducing the impact of bid-ask bounces and stabilizing the analysis for high-frequency trading strategies.

These features are calculated for various time intervals (1 minute, 5 minutes, 15 minutes, etc.) and are fundamental for analyzing price movements, trends, and volatility.

3.2.3. Technical Indicators

Technical indicators offer deeper insights into market dynamics through mathematical modeling:

- **Exponential Moving Average (EMA)**:

$$EMA_t = \alpha P_t + (1-\alpha) EMA_{t-1},$$

  where α is the smoothing factor. EMA gives more weight to recent prices and is critical for identifying trends with reduced lag compared to the simple moving average.
- **Moving Average Convergence Divergence (MACD)**:

$$\text{MACD} = EMA_{\text{fast}} - EMA_{\text{slow}}.$$

  MACD provides momentum-based buy/sell signals and trend strength by analyzing the interaction between short-term and long-term EMAs.
- **Relative Strength Index (RSI)**:

$$\text{RSI} = 100 - \frac{100}{1 + RS},$$

  where $RS = \frac{\text{average gain}}{\text{average loss}}$. RSI helps identify overbought and oversold conditions, critical for timing market entries and exits.
- **Commodity Channel Index (CCI)**:

$$\text{CCI} = \frac{P_{\text{typical}} - SMA(P_{\text{typical}})}{0.015 \cdot \sigma},$$

  with $P_{\text{typical}} = \frac{P_{\text{high}} + P_{\text{low}} + P_{\text{close}}}{3}$. CCI measures price deviations from an average and signals cyclical trends.
- **Directional Movement Index (DMI)**: Uses +DI, -DI, and ADX to quantify the strength and direction of trends.

3.2.4. Volume Features

Volume metrics capture trading activity and liquidity dynamics:

- **Volume (V)**:

$$V = \text{bid\_quantity} + \text{ask\_quantity}.$$

  High trading volume indicates active markets and potential for significant price movements.
- **Volume Ratio**:

$$\text{Volume Ratio} = \frac{\text{bid\_quantity}}{\text{ask\_quantity}}.$$

  This ratio evaluates the balance of buying versus selling pressure.
- **Volume Imbalance**:

$$\text{Volume Imbalance} = \frac{\text{bid\_quantity} - \text{ask\_quantity}}{\text{bid\_quantity} + \text{ask\_quantity}}.$$

  It reveals directional biases in trading activity.
- **Order Count**: Total number of orders, a measure of market participation and interest.
- **Average Order Size**:

$$\text{Avg Order Size} = \frac{V}{\text{order\_count}}.$$

  This provides insights into the dominance of institutional versus retail trading activity.

3.2.5. Volatility Features

Volatility is measured using the Yang-Zhang model, which integrates:

- **Overnight Volatility**:

$$\sigma^2_{\text{overnight}} = \text{Var}(\ln P_{\text{open}} - \ln P_{\text{close, prev}}).$$

- **Intra-day Volatility**:

$$\sigma^2_{\text{intra}} = \text{Var}(\ln P_{\text{close}} - \ln P_{\text{open}}).$$

- **Rogers-Satchell Volatility**:

$$\sigma^2_{\text{RS}} = \text{Var}\left(\ln \frac{P_{\text{high}}}{P_{\text{close}}} - \ln \frac{P_{\text{low}}}{P_{\text{close}}}\right).$$

These measures collectively assess risk, making them essential for portfolio optimization, options pricing, and trading strategies such as volatility breakouts.

3.2.6. Applications

The feature engineering framework supports:

- **Market Microstructure Analysis**: Understanding liquidity, price discovery, and efficiency.
- **Trading Strategy Development**: Strategies like mean reversion, momentum, and breakout trading.
- **Risk Management**: Volatility analysis aids in mitigating exposure.
- **Machine Learning**: Features are directly usable for predictive models in financial analytics.

3.3. The Baseline Model

For our baseline model, we chose an LSTM model as it was described in [stanford], consisting of an LSTM module with 5 layers and a hidden size of 200, projecting it's output through one feedforward projection layer.

3.4. The Main Model(s)

Initially, we planned on comparing four models; namely a baseline LSTM, standard MLPs, KANs with a B-Spline family of functions, and KANs with a Rational family of functions. In practice KANs are atleast twice as slow during inference when compared to regular MLP networks, however leveraging the efficient implementation of rational functions in CUDA, Rational Basis KANs were able to achieve speeds nearly identical to those of traditional MLPs (Yang & Wang, 2024).

Unfortunately, during testing RKANs appeared to break unexpectedly, and this seemed to happen at random. We postulate that this can be traced back to their automatic initializations which were somehow causing divisions by zero which would propogate throughout the network effectively rendering it useless and yielding an output of nan. Since these networks are in their academic infancy, there was and is very little community support surrounding bugs like these, so we had to unwillingly discard this network from our testing phase.

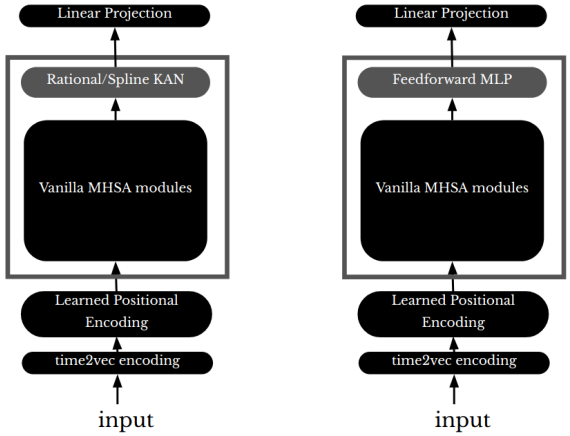

Figure 2. Our decoder-only models compared side-by-side, the input is passed from a time2vec encoding towards a learned positional encoding before being processed by MSHA modules and projected linearly into the output space which is of the form (batch, prediction_horizon)

Our model involves a simple transformer structure with the addition of a time2vec encoding layer before the learned positional encoding layer as suggested by (Muhammad et al., 2023). This time2vec encoding layer is meant to aid the model in capturing the periodic and non periodic nature of our data.

$$\text{t2v}(\tau)[i] = \begin{cases} \omega_i \tau + \varphi_i, & \text{if } i = 0, \\ \sin\left(\omega_i \tau + \varphi_i\right), & \text{if } 1 \leq i \leq k. \end{cases}$$

The layer achieves this by first transforming the time index ($i = 0$) in an affine manner and composing the remaining time-dependent features ($i > 0$) with an affine transformation followed by a periodic function. The parameters of the affine transformation are trainable. In a practical setting, we used the implementation provided on GitHub by (Garcia, 2025). This implementation works by transforming an input of dimensions [batch, seq, dim] into [batch, seq, dim + num_features] where num_features is a hyperparameter that adds in redundant sinusoidal transformations so the model can capture a wider variety of periodic behaviours. This seemed to be an interesting hyperparameter to toy around with, as the training convergences seemed to be extremely sensitive with respect to it. We found that the ideal number for it seemed to be 10 Intuitively speaking, one can imagine that the model attempts an approximation of a Fourier analysis when optimizing the parameters to separate out the pure periodicities of the data we are working with. We wished to analyze a fully trained time2vec layer in more detail but could not make time for such detours.

4. Experimental Results

As expected the baseline's model was the poorest, and perhaps not even worth commenting on. While the model followed the general trend of the market, it failed to capture the detailed intricacies of the market behaviour. More notably, it did not appear to handle sharp declines well, as can be observed by it failing to keep up with falling market prices in 3

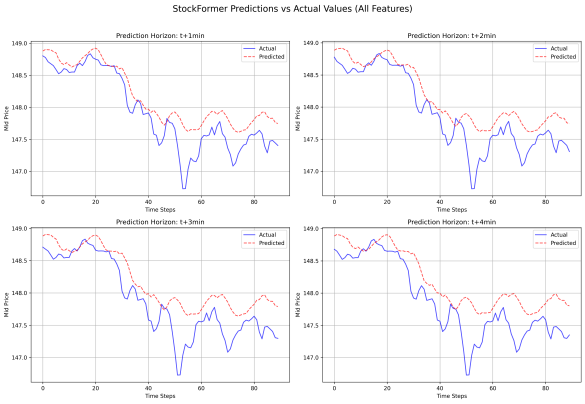

Figure 3. The difference in performance of our baseline LSTM model being benchmarked on minute-by-minute data, having to predict one timestep up till four timesteps into the future. The already poor performance expectedly degrades as we ask the model to predict further into the future.

Moving on to the the spotlights of this article, we compare the performance of B-Spline Basis KANs and MLP based transformers.

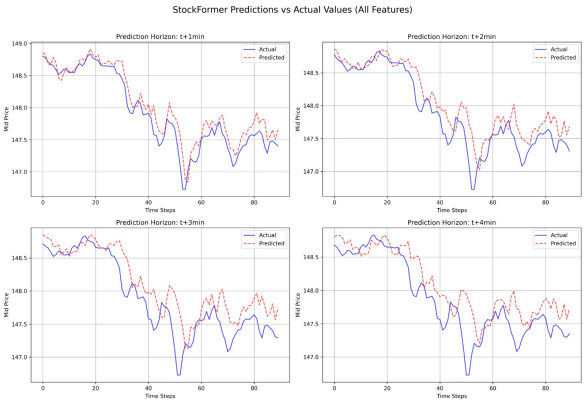

Figure 4. The performance of on MLP model on minute-by-minute data

As can be observed by comparing 4 and 5, we see dissapointing performance from KANs. Although they perform well empirically, MLPs are able to capture market trends in a much better manner. These results were a letdown, for obvious reasons, since KANs had promised us much higher reprentative power, why would they appear to fall so short?

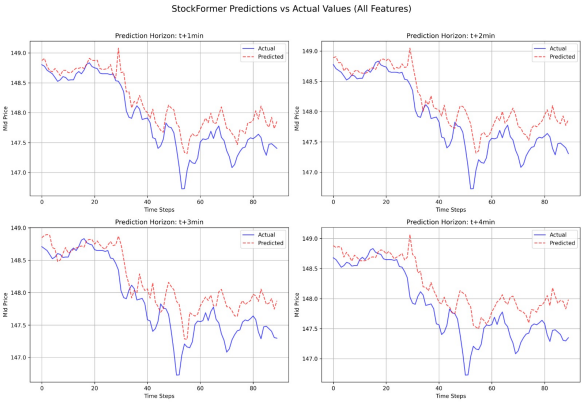

Figure 5. The performance of a KAN model on minute-by-minute data

We observed during the training periods that KANs exhibited extremely erratic convergence, and we postulate that the disappointing performance was partly in due to this unstable training. Compare in the following figures 6 and 7, the training curves for MLPs and KANs,

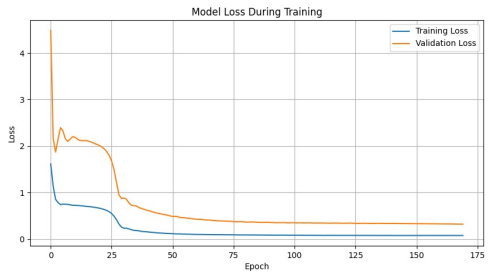

Figure 6. KAN loss curve trained with ADAM

It is to be noted that this behaviour was consistent across a few sample datasets that we tested. So this cannot have been a localized 'speed bump'.

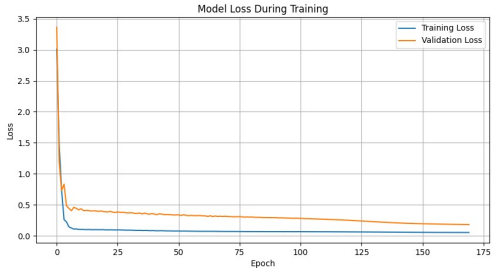

Figure 7. MLP loss curve trained with ADAM

We postulated that this was happening partly because of two reasons; the optimizer of our choice, and the initializations for our KAN. The first conjecture was easy enough to test,

and Lo Behold, as can be observed in 8, we achieved much more stable training when employing SGD with momentum to train our KAN. Although, we're not sure exactly why this happens, we believe that most of the optimizers and initialization algorithms we employ are very MLP-centric, and KANs being a completely new architecture may not be well suited for training using these tools. There is still plenty we do not know about these networks, and chief amongst them, we know nothing about the shape of their loss surfaces, especially when compared to MLPs.

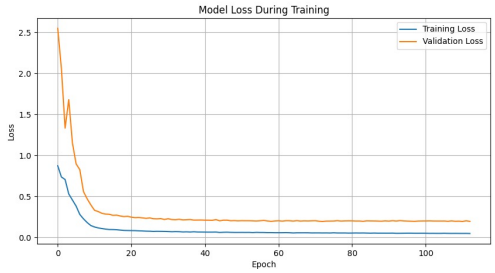

Figure 8. KAN loss curve trained using SGD with momentum

Noting the fact that they appear to converge must faster using SGD likely hints at something important about the shape of their loss curves–although we are not knowledgeable enough to say what exactly. It is also worthy to mention that this behaviour seems to be exhibited in over parameterized models, since training a simple feed forward KAN during initial probing and testing yielded smooth loss curves.

Little can be said about the matter of initializations except the fact that KANs employ a very MLP centric method of initializing their parameters, which may again, be very ill suited considering how differently these networks behave when compared to MLPs. We also highly suspect that initializations were the reason why rational basis KANs kept failed sporadically, but attempts at trying to get to the root of the issue ended unsuccessfully due to constraints of time and schedule.

We also attempted to gauge the performance of these networks on external and unpredictable events, and were pleasantly surprised to find that KANs appeared much better suited when dealing with these conditions.

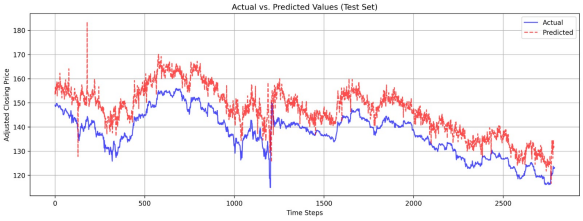

Figure 9. KAN evaluated on the stock market drop during the 2023 earthquake

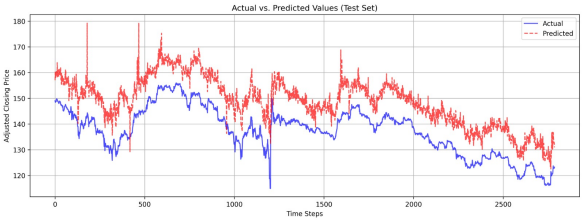

Figure 10. MLP evaluated on the stock market drop during the 2023 earthquake

Observe closely, how the MLP in 10 seems to produce huge spikes during uncertainty, and is unable to follow the stock market closely during the massive dip in equities during the earthquake. Compare this with KANs 9 which follow the general market trend much more closely and conservatively, and even handle the stock market dip reasonably better than MLPs.

Note also that while we had many different variations of the dataset, that is, a minute by minute dataset, an hour by hour dataset, and so on, the results we obtained from training on these various datasets were more or less similar to the ones displayed above, so we found it redundant to acknowledge them to draw the same conclusions.

We also aimed to determine whether time2vec actually had a significant enough affect on our training accuracies, and as the results suggest, it was imperative in achieving excellent accuracies and a smoother convergence.

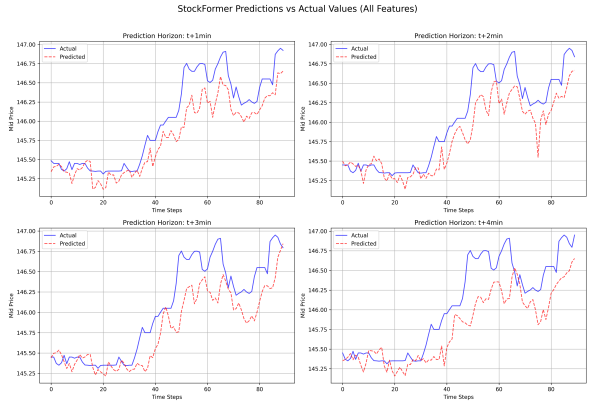

Figure 11. MLP evaluated without time2vec encoding, with essentially num_frequency $= 0$

Compare this with another MLP trained on the same dataset, with time2vec encoding enabled,

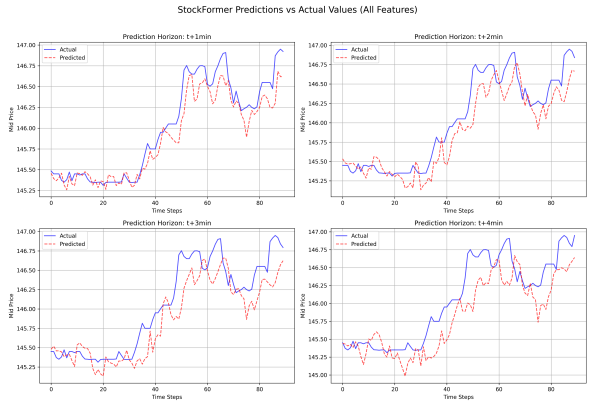

Figure 12. MLP evaluated with time2vec encoding enabled, the model shows considerable improvement over its counterpart with no time2vec. However it is to be noted that these results were obtained with num_frequency $= 10$

. We attempted to increase the hyper parameter to 20, to observe the results, however this greatly crippled our convergence and accuracies; suggesting that the model had begun to be overparameterized

Comparing 11 and 12 helps drive home the fact that time2vec was an essential part of our model architecture and greatly boosted performance.

Trading Strategy

Lastly, we aimed to see if our model would perform, if at all, in a real world scenario. So we constructed a rudimentary trading strategy based on the model's predictions to see if we could strike oil. And indeed, our model was able to make a considerable size of profit (in a simulated environment). Our strategy was as follows:

1. Predict the future price $\hat{P}_{t+h}$ at time $t+h$, where h is the prediction horizon.

2. Compare the predicted price $\hat{P}_{t+h}$ with the current price P_t:

$$\text{If } \hat{P}_{t+h} > P_t, \quad \text{then take a } \textbf{Buy} \text{ position.}$$

$$\text{If } \hat{P}_{t+h} \leq P_t, \quad \text{then take a } \textbf{Sell} \text{ position.}$$

3. Close the position at the prediction horizon $t+h$ by executing the opposite trade:

- For a **Buy** position: Sell at P_{t+h}.
- For a **Sell** position: Buy at P_{t+h}.

Mathematical Representation

Step 1: Predict the Future Price Let $\hat{P}_{t+h}$ be the predicted price at time $t+h$, generated by a prediction model f based on available features $\mathbf{X}_t$:

$$\hat{P}_{t+h} = f(\mathbf{X}_t)$$

Step 2: Trading Decision At time t:

$$\text{Trade Decision} = \begin{cases} \text{Buy}, & \text{if } \hat{P}_{t+h} > P_t \\ \text{Sell}, & \text{if } \hat{P}_{t+h} \leq P_t \end{cases}$$

Step 3: Close Position At time $t+h$, close the position:

$$\text{Profit or Loss} = \begin{cases} P_{t+h} - P_t, & \text{for a Buy position} \\ P_t - P_{t+h}, & \text{for a Sell position} \end{cases}$$

Notation

- $\mathbf{X}_t$: The feature vector at time t, which may include historical prices, technical indicators, and other relevant factors. - $f(\cdot)$: Our prediction model - h: The prediction horizon, representing the number of time steps ahead for the forecast. - P_t: The current price at time t. - P_{t+h}: The actual price at the prediction horizon.

5. Conclusions

We went into this project having high hopes for these novel KAN networks, but were meant with empirical success and theoretical disappointment. We hoped these networks would prove promising enough to garner more research and interest, however as we have discovered first hand, they come with several limitations; speed and general uncertainty about their behaviour being the primary problems facing them. As our professor once said, we tend to get stuck in local minimas with regards to architectures in literature, and these models were an interesting attempt at escaping that minima, however for these networks to have a chance at

replacing MLPs, they do not need to be just a little better, they have to be lightyears ahead, which they do not appear to be. Regardless, they still provide an interesting theoretical playground, and further research should aim at finding better suited tools designed specifically for these networks, and studying their loss surfaces would be a good place to start. We also attempted to enhance our models' performance by incorporating a reinforcement learning loop, and this showed promising results at first. However, given the time constraints, and the nature of our schedules we decided to leave it as a future endevour.

6. Acknowledgements

Both participants played a crucial role in the completion of this project, and complemented each other excellently. Abdul Rehman was more theory oriented, and leaned towards reasoning about the results we recieved, setting up the models, and reading papers. Zindan, on the other hand was more result oriented and an expert in finance, he engineered the features himself, and created the training pipelines.

References

Chen, G., Chen, Y., and Fushimi, T. Application of deep learning to algorithmic trading, 2017. URL https://api.semanticscholar.org/CorpusID:38535712.

Garcia, A. R. time2vec - a pytorch implementation of time2vec, 2025. URL https://github.com/andrewrgarcia/time2vec. Accessed: 2025-01-26.

Liu, Z., Wang, Y., Vaidya, S., Ruehle, F., Halverson, J., Soljačić, M., Hou, T. Y., and Tegmark, M. Kan: Kolmogorov-arnold networks, 2024. URL https://arxiv.org/abs/2404.19756.

Muhammad, T., Aftab, A. B., Ibrahim, M., Ahsan, M. M., Muhu, M. M., Khan, S. I., and Alam, M. S. Transformer-based deep learning model for stock price prediction: A case study on bangladesh stock market. *International Journal of Computational Intelligence and Applications*, 22(03), April 2023. ISSN 1757-5885. doi: 10.1142/s146902682350013x. URL http://dx.doi.org/10.1142/S146902682350013X.

Yang, X. and Wang, X. Kolmogorov-arnold transformer, 2024. URL https://arxiv.org/abs/2409.10594.